



Can you sell novatab? An honest assessment

Short answer: Yes, but not as a *paid Chrome Web Store listing* — that model is mostly dead for extensions. There are two realistic paths that actually fit what you've built, and both are below. This doc is the brutally honest version, including the stuff that will get your submission rejected or your launch ignored if you don't fix it first.

1. What you actually have

- ~14,400 lines of vanilla JS/HTML/CSS (`app.js` 10.3k, `newtab.html` 1.8k, `style.css` 2.3k), no backend, no build step, no dependencies.
- 20+ features: workspaces, bookmarks/notes/tasks, Pomodoro focus mode + site blocking, calendar, journal, habit/mood tracking, kanban, reading queue, tab sessions, analytics, weather, AI command bar + daily briefing + smart-organize (Anthropic), E2E-encrypted Drive sync, theme marketplace, voice capture, omni-search.
- A genuinely good README (setup, OAuth, permissions, security notes all documented).
- 19 commits, started ~7 weeks ago. This is a fast, solo-built, feature-dense project — that's a real asset.

The feature list is *more* than most paid "new tab" products ship with. The AI features, E2E-encrypted sync, and voice capture are genuinely differentiated — most competitors in this space don't have any of the three.

2. The market reality (the honest part)

The "new tab dashboard" category is one of the most saturated on the Chrome

Web Store: **Momentum, Toby, Infinity New Tab, Tabliss, Dayboard, Workona, Vega, Nimble** — several have 1M+ users and have been iterating for 5-10 years. Most are free with a 2 – 5/*moor* 20-30/yr "Pro" tier (extra themes, cloud sync, integrations).

Two consequences:

1. **Feature parity won't get you users.** You're already close to or past parity. The gap isn't features — it's distribution. A brand-new listing with zero reviews and no marketing will get close to zero organic installs, no matter how good the code is.
2. **"Paid Chrome Web Store extension" mostly doesn't exist anymore.** Google deprecated the Chrome Web Store Payments API in 2020. Almost nobody sells extensions as a one-time CWS purchase today — the standard pattern is *free extension + external subscription* (Stripe/Paddle + a license-check backend), which means you need a backend you don't currently have.

None of this means "don't bother" — it means **don't build for the wrong model.**

3. Monetization paths, ranked by realism for *this* codebase

Option A — Sell it as source / a white-label template (fastest, lowest effort)

Sell the codebase itself to developers, freelancers, and agencies who want to ship a branded "new tab" extension for a client or their own product, on marketplaces like **CodeCanyon, Gumroad, or Flippa**. Buyers get a polished, feature-complete, no-backend, no-dependency codebase with a real README — that's exactly what these marketplaces reward.

- **Effort:** low — mostly packaging, docs, and a demo.
- **Realistic price:** 39–149 per license (one-time), depending on marketplace and how much "white-label" polish you add (rebrand-in-5-minutes guide, swap-

the-logo script, etc.).

- **Revenue:** a few hundred to a few thousand dollars over time, long-tail. Not life-changing, but it's basically "package what you already built."
- **Why it fits:** no backend, no ongoing support burden, no Chrome Web Store review risk.

Option B — Free extension on CWS + paid "Pro" subscription (the standard model, but needs a build-out)

Ship the extension free under your own brand, build an install base, then gate premium features (AI without needing your own Anthropic key, cross-device sync beyond Drive, premium themes/wallpapers, priority support) behind a Stripe subscription validated by a small backend.

- **Effort:** medium-high — needs a backend (a single Cloudflare Worker or Vercel function is enough), Stripe integration, a license/entitlement check in `app.js`, and **real marketing** to get installs.
- **Realistic revenue (year 1, no ad spend):** likely 0–300/mo unless you actively market it (ProductHunt, Reddit, X, YouTube reviews of "best new tab extensions"). With marketing effort, \$500-2000/mo is achievable for niche productivity extensions with a loyal audience.
- **Why it fits, eventually:** your AI features are the natural premium hook — but only once you remove the "bring your own API key" friction (see §5).

Option C — Direct paid CWS listing

Not recommended. The infrastructure for this is effectively gone. Skip it.

Option D — Free + portfolio/resume value

Even if neither A nor B makes meaningful money, shipping this free on the CWS under your name is a strong portfolio piece — "10k+ users, 4.x★, self-built AI-powered productivity extension" is a great line on a resume/LinkedIn, and it's the cheapest way to validate whether Option B is worth pursuing later.

My recommendation: do A and D in parallel now (both are cheap), and

treat B as a "if D gets traction" follow-up — don't build subscription infrastructure before you know anyone wants it.

4. What will block or sink a Chrome Web Store submission *right now*

These aren't nice-to-haves — they are things Google's review process actively checks, especially for "new tab override" extensions (a category with a long history of malware abuse, so it gets extra scrutiny):

Issue	Why it matters	Severity
No privacy policy	Required for any extension requesting <code>tabs</code> , <code>history</code> , <code>bookmarks</code> , <code>geolocation</code> , <code>identity</code> / <code>identity.email</code> , <code>downloads</code> . Missing policy = automatic rejection.	 Blocker
Very broad permission set (10 permissions + 10 host permissions)	"New tab override" extensions with many permissions get manually reviewed and frequently get a "please justify each permission" request or rejection on first submission.	 Blocker / delay
No LICENSE file	If you're selling this (Option A) or want "all rights reserved," you need an explicit license — right now anyone who finds the repo has an ambiguous right to reuse it.	 Important
Hardcoded <code>GOOGLE_CLIENT_SECRET</code> in <code>app.js</code> (<code>GOCSPX-...</code>)	Works for "installed app" OAuth (Google treats this as non-confidential, PKCE protects the flow), but if this extension scales to real users under your brand, every install shares one OAuth client's quota/branding and the secret sits in plaintext source. Switching to Google's	 Should fix before scaling

Issue	Why it matters	Severity
	"Chrome Extension" OAuth client type removes the need for a client secret entirely and ties the client to your extension ID.	
OAuth consent screen verification	<code>drive.appdata</code> is a non-sensitive scope, but <code>userinfo.email</code> / <code>userinfo.profile</code> plus an "External" OAuth app with real users will require Google's basic verification (privacy policy URL + verified homepage). Budget a few days for this once you publish.	 Needed before wide release
"novatab" naming	Worth a quick trademark/CWS-namespace check — "Nova"-prefixed productivity apps are common; pick a name you can actually defend/brand if this becomes a product.	 Cheap to check now
No tests	Not a CWS blocker, but if you're selling this (source or subscription), "no test suite" is a credibility/quality signal that matters to buyers and to your own ability to ship updates without regressions.	 Quality issue
AI features need user's own Anthropic API key	Good for <i>your</i> costs (zero), bad for <i>conversion</i> — most non-technical users will never get an API key, so your most differentiated features (Ask AI, daily briefing, smart organize) will go unused by ~95% of installs. This undercuts the premium pitch.	 Strategic, fix if pursuing Option B

5. What to improve, in priority order

1. **Pick the model first.** Option A (sell as source) needs almost none of the below except LICENSE + packaging. Option B (free + Pro) needs all of it. Don't do CWS-submission work for a model you haven't committed to.
2. **Write and host a privacy policy.** A static page on GitHub Pages is sufficient and free. Cover: what data is collected (none, beyond what `chrome.storage.local` holds on-device), what Drive sync does, that the AI key is user-supplied and never leaves the device except to `api.anthropic.com`. Most of this is already written in your README's "Security Notes" — it just needs to be a standalone hosted page linked from the CWS listing.
3. **Audit and document every permission.** For each of `tabs`, `bookmarks`, `history`, `downloads`, `storage`, `topSites`, `identity`, `identity.email`, `geolocation`, `declarativeNetRequest`, write one sentence for the CWS listing's permission justification field. Consider whether any can be made optional (`optional_permissions`) and requested only when the relevant feature is first used — this alone meaningfully reduces review friction.
4. **Add a LICENSE.** If selling source (Option A), use a proprietary/commercial license with clear redistribution terms. If going free+open (Option D), MIT/AGPL are fine — but pick one deliberately, don't leave it blank.
5. **Switch the Google OAuth client to the "Chrome Extension" type** (no `client_secret`, bound to your extension ID). This is a ~30 minute Google Cloud Console change plus a small `app.js` edit, and removes a real liability before you have real users.
6. **If pursuing Option B:** build a thin proxy backend (single Cloudflare Worker) for the AI features with a generous free-tier rate limit funded by you, and gate higher limits / extra AI features behind a Stripe-checked entitlement. This turns "AI command bar" from a feature 5% of users can use into your actual premium hook.
7. **Onboarding polish.** A first-run tour or sample data (a pre-populated "Welcome" workspace with example bookmarks/notes/tasks) makes a huge difference in first-session retention for dashboard-style extensions — right now a fresh install is presumably an empty shell.
8. **Marketing assets.** CWS listing needs: a 1280×800 promo image, a 440×280 small tile, and 3-5 screenshots showing the actual UI. None of this exists yet

and it's the single highest-leverage thing for organic CWS search ranking.

9. **Cross-browser smoke test** on Edge and Brave (both Chromium, both in your README's supported list) — five minutes, catches anything Chrome-specific.
 10. **Submit free to CWS first** (Option D), even before deciding on B. Real install/review data is the cheapest way to find out if a Pro tier is worth building.
-

6. Realistic expectations

Be honest with yourself about numbers:

- **Source/template sale (Option A):** A handful of sales per month at 39 — 149 *once listed on a marketplace with decent SEO—realistically* * 100-500/mo** after the first few months, possibly less, possibly a one-time spike if it gets featured.
- **Free extension, no marketing (Option D):** Single-digit to low-hundreds of installs/month organically from CWS search. This is normal — it took the big players years and (in some cases) paid acquisition to get to scale.
- **Subscription (Option B), year 1, no ad spend:** Likely 0 — 300/mo * *without active marketing (Product Hunt launch, Reddit/r/productivity posts, a YouTube channel)* * **1k-5k/mo MRR**, but that's 12-18 months of consistent work, not a launch outcome.

Bottom line: the code is good enough to be the *product*. The thing standing between you and revenue is distribution and a couple of CWS-compliance gaps — not features. Don't add more features before fixing the privacy policy / permissions / licensing gaps and picking a model.